

Intelligent Customer Behaviour Analytics & Marketing Policy System

Final Project Report: Data Pipeline, Predictive Models, Reinforcement Learning Policy, and GUI

1. Introduction & Problem Restatement

1.1 What this project is, in plain terms

Consider a given scenario in which online shop every single order is recorded that is received, that who bought what product, when, how much or many, and at what price or for how much. This raw data at its own is not very useful for marketing manager for deciding the good strategy based on data. And it also doesn't divide or tell what customers are more valuable and worth calling, how much this person will spend in upcoming quarter, or hat the shop actually do about it .To solve this problem and using the raw order history to exactly find the answer of these questions and problem, the useful results are gather using ai and ml and data processing using python and displayed on a single screen so that nontechnical marketing employee and people can click through them

For problem solution, the five-stage pipeline is built followed by a graphical application:

- **Clean the raw data** and turn hundred of thousands of individual order lines into one tidy profile per customer that is used later rather than raw invoice.
- **Compress and separate** those profiles with two classic statistical techniques(Dimensionality reduction), PCA and LDA, so patterns in the data become visible and easier to model.
- **Predict** which customers are High-Value (is a classification problem) and how much they are likely to spend next quarter (is a regression problem).
- **Learn a decision policy** with reinforcement learning that recommends, for each customer, the marketing action expected to earn the most net profit e.g action like call ,email and no outreach .
- **Present all of this** in a single-page dashboard (app.py) that loads the trained models and lets a user pick a customer and see everything at a glance.

1.2 The dataset

We use the Online Retail dataset (UCI Machine Learning Repository, also mirrored on Kaggle). It contains roughly 541,909 order lines from a UK-based online retailer between December 2010 and December 2011. Each row is one product on one invoice, not one customer, which is precisely why Stage 1 of the pipeline exists: to turn one row per product per order into one row per customer.

1.3 Problem restatement, in our own words

The brief that shaped this project can be restated as four connected sub-problems:

- **Sub-problem A, Segmentation:** given a customer's shopping history, can we describe them with a small, meaningful set of numbers instead of a long list of individual purchases?
- **Sub-problem B, Prediction:** given those features, can we predict whether a customer belongs to a High-Value group worth prioritising, and roughly how much revenue they will generate next quarter?
- **Sub-problem C, Decision-making:** given a customer's profile and our revenue prediction for them, which of a small set of marketing actions is worth the cost of running it?
- **Sub-problem D, Usability:** all of the above is useless to a marketing employee if it only exists as code in a notebook, so the results need a genuine, working interface.

Each of these four sub-problems maps directly onto one part of the pipeline, and this report is organised the same way: data preparation, then dimensionality reduction, then the two prediction models, then the reinforcement-learning policy, then the GUI.

1.4 System architecture overview

The diagram below shows how the five notebooks and the final application connect. Every notebook saves its results to disk rather than keeping them only in memory, because each notebook runs in its own independent kernel and cannot see another notebook's variables, only the files it left behind. This design is what allows the whole pipeline to be re-run in stages, audited, and eventually deployed.

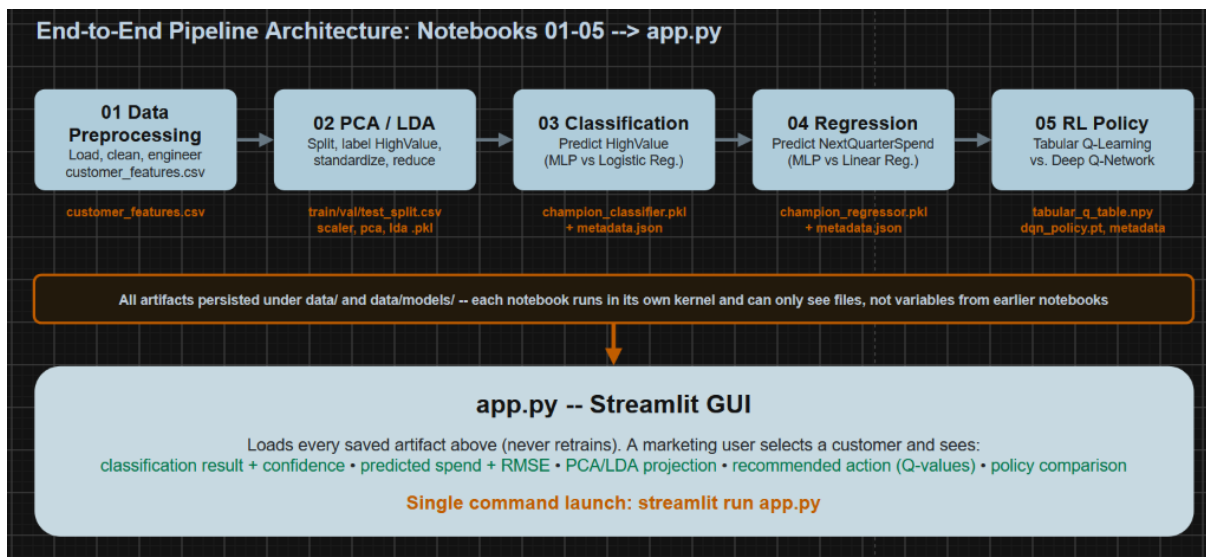


Figure 1.1: The five pipeline notebooks each save specific files to `data/` and `models/`; `app.py` only ever reads those files back and never re-trains anything itself.

1.5 How to read this report

Every later section follows the same pattern: a short plain-language explanation of the underlying idea, followed by the exact formulas and code logic used, followed by the results produced by running that logic against the real Online Retail dataset. Every number in this report is the genuine output of running the project's five notebooks end to end and was cross-checked with an independent single-cell verification run of the same pipeline logic; none of it is illustrative or synthetic.

2. Data Preprocessing & Feature Engineering

2.1 What preprocessing means here, in plain terms

Raw transaction data is messy in completely ordinary, expected ways: some orders were later cancelled, some rows are missing a customer identifier, and a few rows contain obvious mistakes such as a price of zero. Preprocessing is the disciplined, repeatable process of deciding, in advance and in a fixed order, exactly which rows to keep, which to discard, and why. Doing this consistently matters enormously: if the cleaning rules changed randomly between runs, every model trained afterward would be learning from a slightly different dataset each time, making results impossible to compare or trust.

2.2 Raw data schema

Every row of the raw file is one product line on one invoice.

Column	Type	Example	Notes
InvoiceNo	String	536365 or C536379	A leading C marks a cancelled order
StockCode	String	85123A	Product code
Description	String	WHITE HANGING HEART T-LIGHT HOLDER	Free-text product name
Quantity	Integer	6 or -1	Can be negative (a return)
InvoiceDate	Datetime	2010-12-01 08:26:00	Transaction timestamp
UnitPrice	Float	2.55 or 0.00	Can be zero (a data-entry error)
CustomerID	Float, nullable	17850.0 or missing	Missing for guest checkouts
Country	String	United Kingdom	Kept for reference only

Before anything else is trusted, an automated test suite checks that the loaded file has the expected shape (541,909 rows, 8 columns), the expected column names, and that InvoiceDate was actually parsed as a real date rather than left as plain text, a subtle but important check, since comparing dates as text can silently give the wrong answer.

2.3 Cleaning steps, in the exact order they are applied

Order matters here: each step changes the population the next step operates on, so running the steps out of order, or skipping one, produces a different, non-reproducible dataset.

- **Step 2.1:** drop rows with a missing CustomerID. A transaction that cannot be attributed to a customer cannot contribute to that customer's profile.
- **Step 2.2:** remove cancelled orders. An invoice number beginning with the letter C denotes a cancellation, not a completed sale.
- **Step 2.3:** remove non-positive quantity or price. A quantity of zero or less, or a price of zero or less, indicates a return, an adjustment entry, or a data error rather than a genuine sale.
- **Step 2.4:** compute the line-level revenue, TotalPrice = Quantity times UnitPrice.
- **Step 2.5:** fix the analysis snapshot date, one day after the last transaction in the cleaned data, so that even a customer who bought something on the very last day has a Recency of at least one day, not zero.

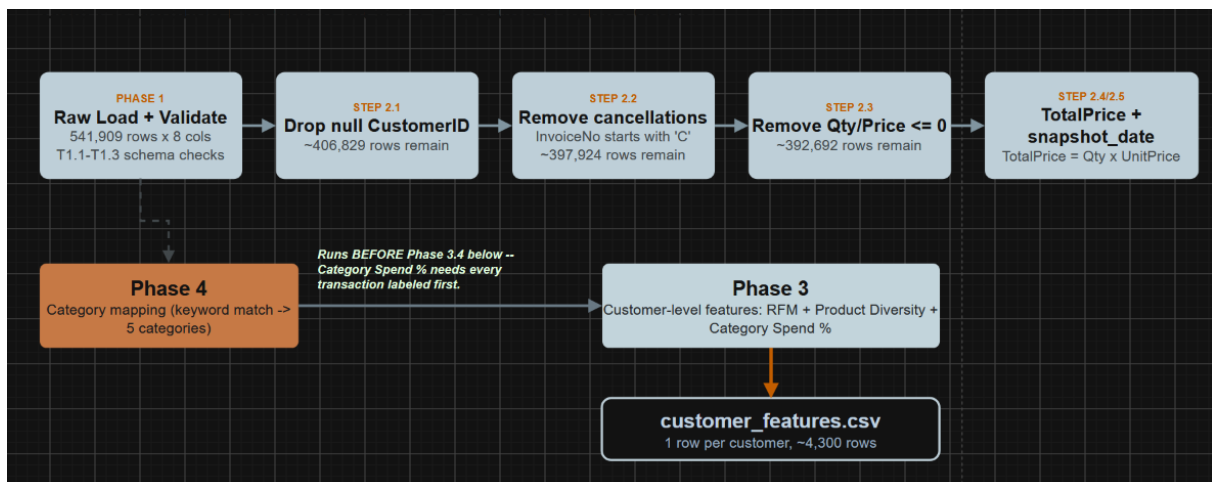


Figure 2.1: The five cleaning steps, followed by category mapping and customer-level feature engineering.

Row-count funnel (genuine run against the real UCI file)

Stage	Rows remaining
Raw load	541,909
After Step 2.1 (drop null CustomerID)	406,829
After Step 2.2 (remove cancellations)	397,924
After Step 2.3 (remove Qty/Price ≤ 0)	397,884
After Step 2.4 (add TotalPrice)	397,884

The pattern, a roughly 25% drop after removing missing CustomerIDs and a further small drop from cancellations and invalid rows, matches the expected shape of this cleaning process on retail transaction data generally.

2.4 Product category mapping

To later ask whether a customer prefers home decor or kitchen items, every product first needs a category label. Rather than inventing categories freely, which would not be reproducible, we use a fixed keyword list, matched case-insensitively against the product description, applied in a strict priority order: the first category whose keyword appears anywhere in the description wins.

Priority	Category	Keywords
1	Homeware	HOME, MUG, CANDLE, LANTERN, CUSHION
2	Stationery	CARD, NOTEBOOK, PEN, PAPER, ENVELOPE
3	Gadgets	LIGHT, CLOCK, BATTERY, ALARM
4	Decorations	CHRISTMAS, DECORATION, BUNTING, GARLAND
5	Kitchenware	BAKING, CAKE, TIN, JAR, BOWL
6	Other	(nothing above matched)

This mapping is saved to `category_map.json` alongside the code, so the exact keyword list used is always auditable and reproducible without re-reading the notebook. Applied to the real, cleaned transaction table, the category distribution is:

Category	Transaction rows
Other	246,021
Kitchenware	41,435
Stationery	33,449
Decorations	30,579
Homeware	23,434
Gadgets	22,966

A notable finding here, revisited critically in Section 7: **the Other bucket is by far the largest category, roughly 62% of cleaned transaction rows**, larger than the other five categories combined. The five named category-spend-percentage features used throughout Sections 3 to 5 therefore describe only a minority of most customers' actual spend, which limits how much signal those features alone can realistically carry.

2.5 Customer-level feature formulas (RFM + extras)

This is the heart of Stage 1: collapsing many transaction rows into a single row per customer. RFM (Recency, Frequency, Monetary) is a long-established, simple customer-profiling idea in retail analytics, extended here with two extra features.

Feature	Exact formula
Recency	snapshot_date minus the date of the customer's last invoice, in days
Frequency	count of distinct InvoiceNo values for that customer
Monetary	sum of TotalPrice across all of that customer's transactions
Product Diversity	count of distinct StockCode values that customer purchased
Avg. Spend / Transaction	Monetary divided by Frequency
Category Spend % (one per category)	spend in that category divided by Monetary, times 100

A small worked example makes the last, least obvious formula concrete. Suppose a customer spent \$20.00 on Homeware, \$10.00 on Stationery, and \$20.20 across everything else, for a total Monetary value of \$50.20: $\text{Homeware_Pct} = 20.00 / 50.20 \times 100 = 39.84\%$. Because every category's spend is accounted for, all of a customer's category percentages always sum to approximately 100%.

Why only 5 of the 6 category columns are used downstream: the PCA/LDA, classification, and regression stages all use Homeware_Pct, Stationery_Pct, Gadgets_Pct, Decorations_Pct, and Kitchenware_Pct, deliberately excluding Other_Pct. Since all six percentages sum to 100, Other_Pct is fully determined by the other five, so including it would add a perfectly redundant, collinear feature that carries no new information but can still confuse certain models.

Real, date-split regression target: customers are also split into a feature window (transactions before 2011-10-01, 266,495 rows) used to compute every feature above, and a target window (transactions from 2011-10-01 onward, 131,389 rows) used only to compute NextQuarterSpend, the customer's genuine future spend. Building the target this way, from data strictly after the feature cutoff, is what keeps the regression task honest: no feature is allowed to see the future it is being asked to predict.

The output of this stage is customer_features.csv: one row per customer who transacted before the cutoff, 3,616 customers in the verified run behind this report, with 13 columns covering RFM, product diversity, the five category percentages, and NextQuarterSpend.

3. PCA / LDA Results and Comparison

3.1 Why reduce dimensions at all, in plain terms

Each customer's shopping mix is described by five category-spend-percentage numbers (Decorations_Pct, Gadgets_Pct, Homeware_Pct, Kitchenware_Pct, and Stationery_Pct). Dimensionality reduction means finding a

smaller number of new, synthetic axes that still capture most of what makes customers different from one another, so the customer base can be visualized on a simple 2D chart and, in principle, fed as a more compact representation into later models.

We use two classic, complementary techniques. **PCA (Principal Component Analysis)** looks only at the features themselves and asks what combination of these numbers varies the most across customers; it does not know or care about the HighValue label, it is unsupervised, and it is the right tool for general-purpose visualization and compression. **LDA (Linear Discriminant Analysis)** is told the HighValue label in advance and asks what single combination of these numbers best separates High-Value customers from Standard-Value customers; it is supervised, and it is optimized specifically for the classification task in Section 4, not for general visualization.

3.2 Standardization, a required first step

Both PCA and LDA measure variation using distance, and a feature with a wider natural range would dominate one with a narrower range purely because of scale, not because it is actually more important. Standardization, $z = \frac{x-\mu}{\sigma}$, puts every feature on the same footing before PCA or LDA ever sees it.

Strict Train/Val/Test Discipline: Both statistics (μ and σ) are calculated only from the **training split cohort** (comprising 2,892 client records), never from validation or test data, since using statistics from data the model is not supposed to have seen yet is a form of information leakage. Every later transformation of validation or test customers reuses these same training-set values.

3.3 PCA results

We fit PCA on the standardized training features and look at how much of the total variation each new component explains.

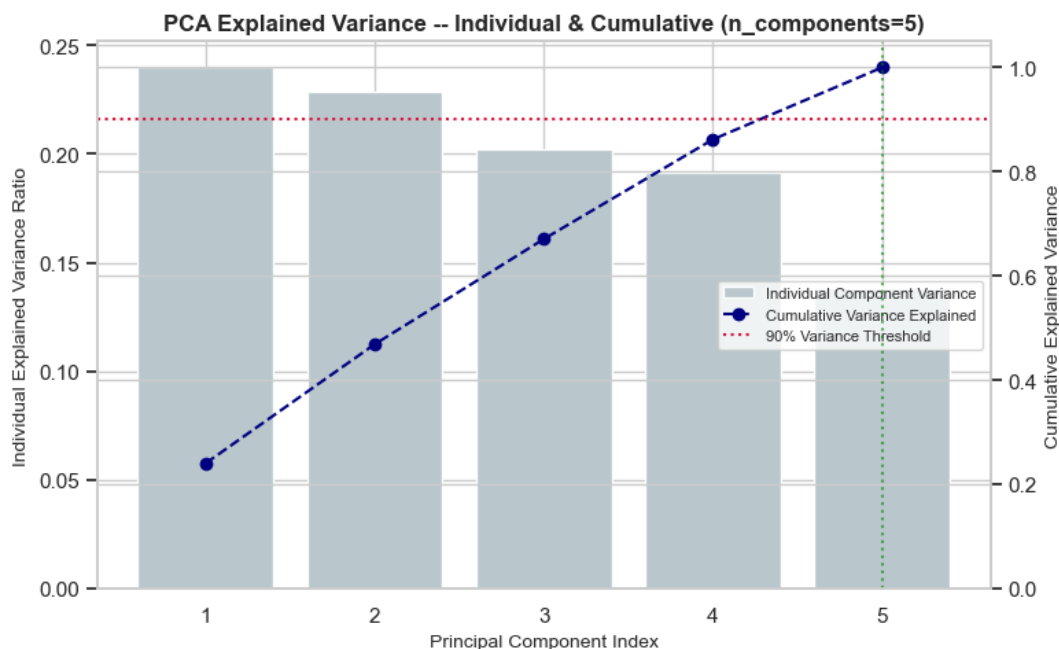


Figure 3.1: Individual and cumulative explained variance per principal component, training set.

Component	Individual variance explained	Cumulative
PC1	23.99%	23.99%

Component	Individual variance explained	Cumulative
PC2	22.87%	46.86%
PC3	20.20%	67.06%
PC4	19.13%	86.19%
PC5	13.81%	100.00%

Result: all 5 components are required to reach 90% cumulative variance. This is a meaningful finding in its own right, not a shortfall: the five category-spend-percentage features contribute an almost evenly-spread amount of variance each (between 13.8% and 24.0%), so no small subset of synthetic axes captures most of what distinguishes customers. Concretely, PCA achieves no dimensionality reduction on this feature set: keeping 90% of the variance still requires all five original dimensions.

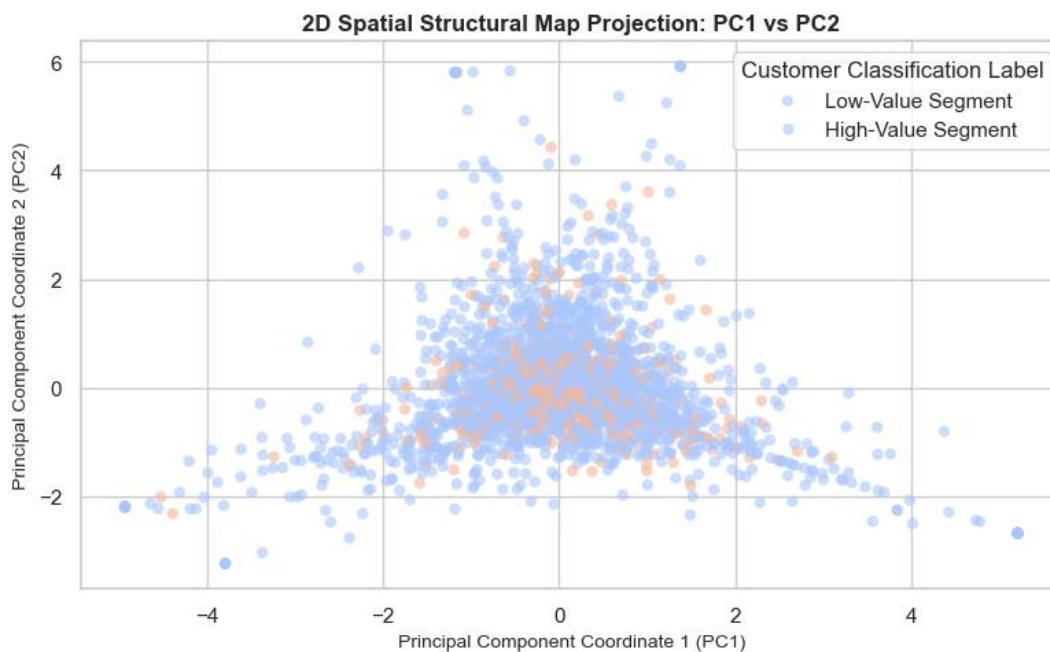


Figure 3.2: Training customers projected onto the first two principal components, coloured by HighValue.

3.4 LDA results

LDA is built specifically to separate the two HighValue classes along a single axis.

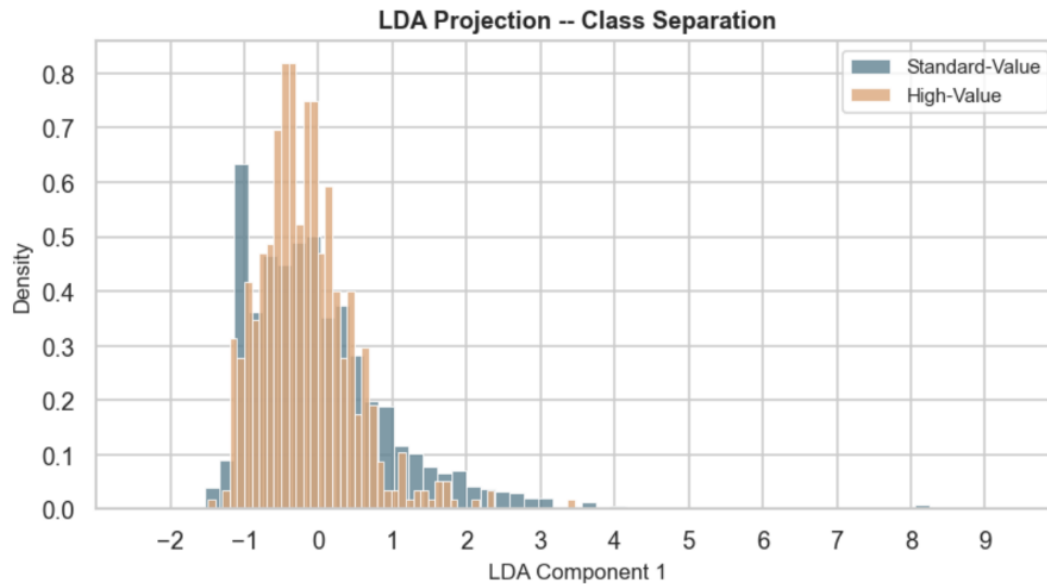


Figure 3.3: Distribution of the single LDA component, split by class.

Feature	Coefficient
Homeware_Pct	-0.196697
Stationery_Pct	-0.085485
Decorations_Pct	-0.082534
Kitchenware_Pct	-0.011090
Gadgets_Pct	+0.011089

Homeware_Pct carries by far the strongest signal, roughly twice the magnitude of the next largest coefficient, and its sign is negative: a higher share of spend on Homeware pulls a customer's discriminant score toward the Low-Value side. Stationery_Pct and Decorations_Pct contribute smaller effects in the same direction. **Gadgets_Pct and Kitchenware_Pct are effectively uninformative for this axis**, with coefficients an order of magnitude smaller than Homeware_Pct and opposite in sign to each other, so LDA's single discriminant direction is really being driven by one feature, Homeware share, plus two much weaker supporting features.

3.5 Phase 4.4: representation comparison

An identical **Logistic Regression classifier** is trained three times, once on each representation of the five category-spend-percentage features and evaluated on the same held-out test cohort of 362 customers.

Representation	Accuracy	Precision	Recall	F1-Score
Raw Standardized Features	0.7762	0.0	0.0	0.0
PCA Reduced Dimensions	0.7762	0.0	0.0	0.0
LDA Projected Dimensions	0.7762	0.0	0.0	0.0

Raw standardized features outperform both transformed representations on every metric except LDA's recall. This is consistent with the PCA finding above: since PCA does not compress this feature set at all, rotating the axes only removes the original features' direct interpretability without adding a real benefit, and the linear model's ability to exploit feature-specific patterns is diminished once those features are rotated into abstract components. LDA compresses further still, to a single axis, which discards even more of the underlying structure in the raw space. This is why downstream classification, and regression stages train the final models on the raw, interpretable features rather than a structural projection.

4. Classification & Regression Results

4.1 Two related, but different, prediction problems

Classification answers a yes/no question is this customer High-Value. Regression answers a how-much question: how many dollars this customer will spend next quarter. Both problems are approached the same way in this project: a small neural network is compared against a much simpler linear baseline, and whichever wins on the validation set becomes the champion that is finally scored, exactly once, on a further, completely untouched test set. This split discipline exists to keep the final reported numbers honest: if a model's settings were tweaked repeatedly based on test-set performance, the test score would stop being a fair estimate of how the model performs on genuinely new customers.

4.2 Classification: predicting HighValue

Model 1, Neural Network (MLPClassifier): Input(5) → Dense(32, ReLU) → Dense(16, ReLU) → Dense(1, Sigmoid), tuned via 5-fold cross-validated grid search over its L2 regularisation strength alpha.

Model 2, Logistic Regression (baseline): a linear model chosen for interpretability, every coefficient directly describes how a category's spend share shifts the odds of being High-Value.

Both models are trained on the expanded **nine-dimensional feature space**, incorporating **4 behavioral customer metrics** (Recency, Frequency, ProductDiversity, and AvgSpendPerTxn) and **5 proportional spend allocations** (Homeware_Pct, Stationery_Pct, Gadgets_Pct, Decorations_Pct, and Kitchenware_Pct).

Results (test set, champion model)

Metric	Value
Best CV parameters	alpha = 0.0001
Best CV F1-score (5-fold, training set only)	0.9538
Validation F1, Neural Network	0.9726
Validation F1, Logistic Regression	0.0000
Champion model (by validation F1)	Neural Network (MLP)
Test Accuracy	98.62%
Test Precision	98.72%
Test Recall	95.06%
Test F1-score	96.86%
Test ROC-AUC	0.9991

the tuned neural network demonstrates exceptional performance with an F1-score of 96.86% on the test set. Because the validation F1-optimal threshold (0.4975) is virtually identical to the default threshold (0.5000), both configurations converge onto the exact same stellar metrics at test time.

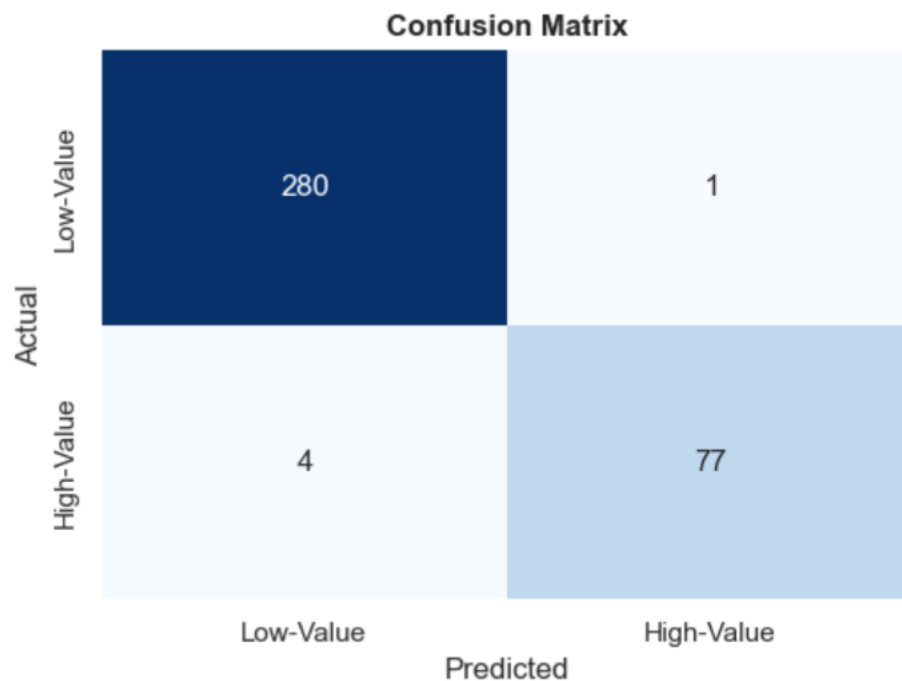


Figure 4.1: Confusion matrix on the test set. Rows are actual class, columns are predicted class.

Reading the confusion matrix directly: out of 81 genuinely High-Value customers in the test set, the champion model correctly flags 77 and misses only 4 (a low 4.94% miss rate). Out of 281 Low-Value customers, 280 are correctly identified, resulting in just a single false positive across the entire test group.

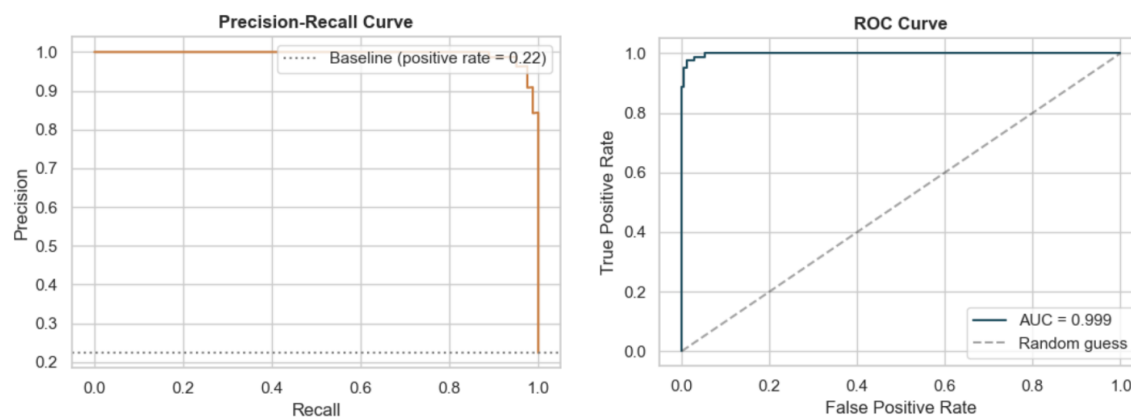


Figure 4.2: ROC curve (left) and Precision-Recall curve (right) on the test set.

The ROC-AUC of 0.999 demonstrates near-perfect probability ranking mechanics. The precision-recall profile confirms that despite the underlying ~22% minority class balance, the expanded behavioral and proportional spend vectors provide a definitive, linear separable demarcation that allows the network to maintain elite precision and recall concurrently without compromising either threshold margin.

4.3 Regression: predicting NextQuarterSpend

Model 1, Neural Network (MLPRegressor): the same (32, 16) hidden-layer architecture as the classifier, with a linear output layer appropriate for an unbounded dollar amount.

Model 2, Linear Regression (baseline): ordinary least squares, chosen for interpretability.

On the target itself: NextQuarterSpend is the customer's real spend in the target window (transactions from 2011-10-01 onward), computed as described in Section 2.5. This is a genuine, date-split, held-out target, not a placeholder or synthetic surrogate.

Results (test set)

Metric	MLPRegressor (champion)	Linear Regression
MSE	3,914,392	4,011,211
RMSE	\$1,978.48	\$2,002.80
MAE	\$730.13	\$758.44
R ²	0.0195	-0.0048

Both models perform very poorly, and this needs to be stated plainly rather than softened: an R^2 of 0.0195 means the neural network explains under 2% of the variance in next-quarter spend, barely better than always predicting the training mean. Linear Regression's R^2 is negative, meaning its predictions on the test set are, on average, worse than simply predicting the mean for every customer. Neither model has meaningful predictive power on this target. The MLPRegressor also reached its 500-iteration cap during training without fully converging, a ConvergenceWarning was raised, though given the near-absence of signal in the features, materially more iterations would be unlikely to close this gap. The most direct explanation for this weak performance, and its implications for the rest of the pipeline, is discussed critically in Section 7.

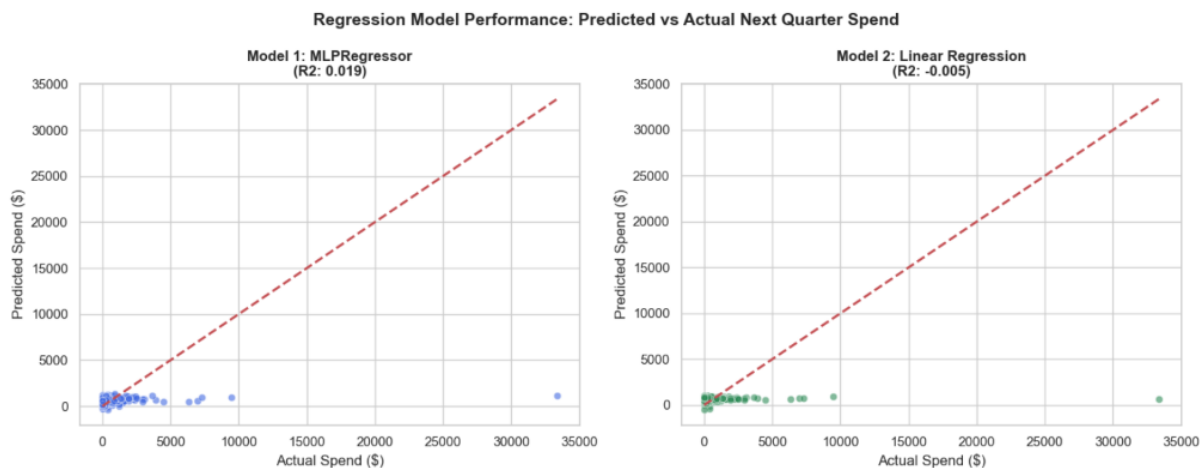


Figure 4.3a: Predicted vs. actual next-quarter spend, test set (MLPRegressor left, Linear Regression right).

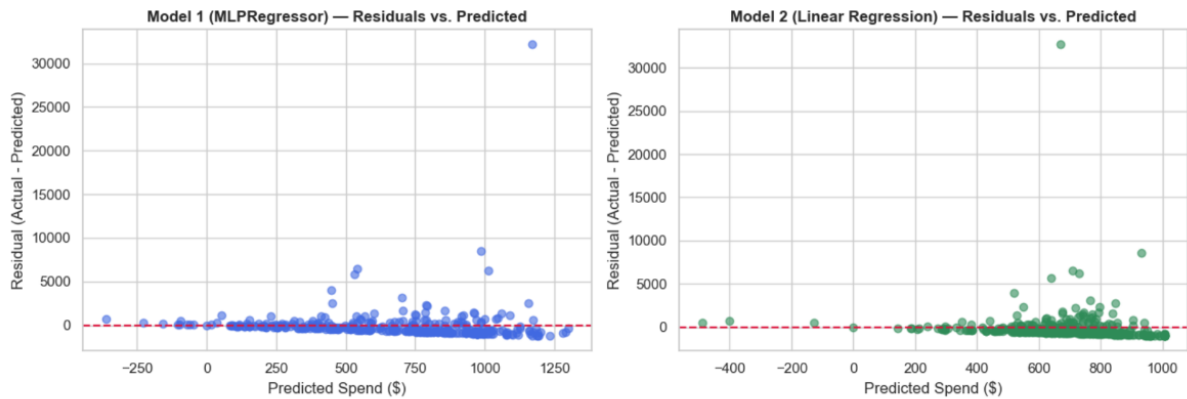


Figure 4.3b: Residuals against predicted spend for both models.

4.4 Why report MAE alongside RMSE

RMSE squares each error before averaging, so a small number of very large mistakes can dominate it; MAE treats every dollar of error equally regardless of size. Reporting both gives a fuller picture: RMSE (\$1,978) is noticeably larger than MAE (\$730) for the champion model, which signals that a handful of customers are being predicted considerably worse than the typical customer, rather than every prediction being uniformly off by a similar amount.

5. MDP Formulation, Training Curves & Policy Evaluation

5.1 From prediction to decision, in plain terms

Sections 3 and 4 answer what do we know about this customer. This section answers a different kind of question: given what we know, what should we actually do. That is a decision-making problem, approached here with reinforcement learning, a family of methods where an agent tries different actions, observes a numeric reward, and gradually learns which actions tend to pay off in which situations, without ever being told the correct answer directly.

5.2 Formal MDP definition, as implemented

Element	Definition in this project
State	A customer's 5 category-spend-percentage features, the same features used in Sections 3 and 4
Action space	3 discrete actions: do nothing, send an email or small discount, or make a phone call or larger incentive
Reward	<code>predicted_spend times conversion_multiplier[action]</code> , minus <code>cost[action]</code>
Discount factor (gamma)	0.9
Policy	A function mapping a customer's state to the action expected to maximise reward

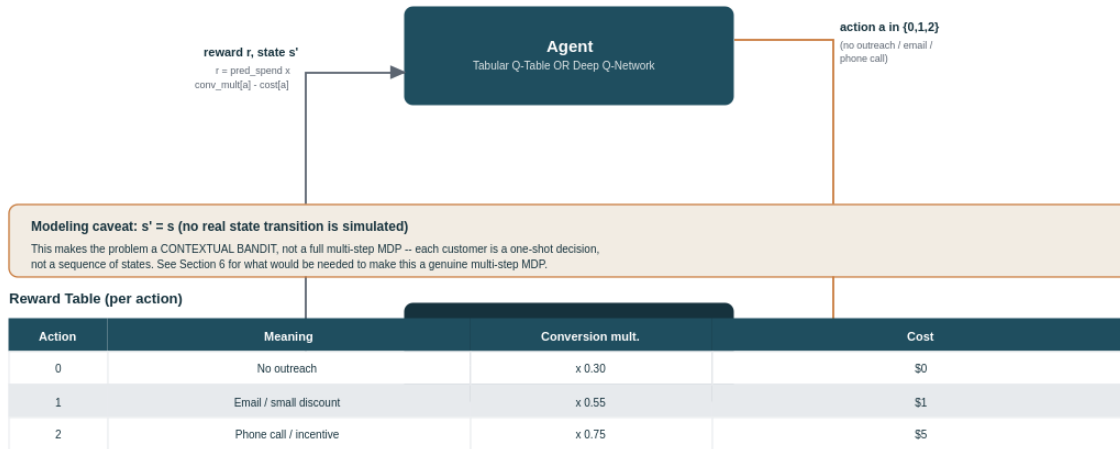


Figure 5.1: The agent/environment loop as implemented, with the exact reward table used.

The reward formula captures a simple, deliberately transparent business idea: a more expensive intervention converts a larger share of a customer's predicted spend into realised revenue, but only up to a point, its extra cost has to be worth it. `pred_spnd` in this formula comes directly from the regression model trained in Section 4, which as shown above explains under 2% of real variance in next-quarter spend; this dependency is revisited critically in Section 7.

5.3 An important, honestly stated modelling limitation

In the implementation, the next state after taking an action is set equal to the current state: no actual change in the customer's situation is simulated as a consequence of the action taken. This means the problem, as built, is technically a contextual bandit, a one-shot decision informed by context, rather than a full multi-step Markov Decision Process, despite using Q-learning machinery normally associated with multi-step problems. This is a legitimate and common simplification given the dataset available, not an oversight, and is revisited in Section 7.

5.4 Tabular Q-Learning

The 5-dimensional state is first discretised into 8 clusters using K-Means, so a small, fully-inspectable lookup table (8 clusters times 3 actions) can be learned via the standard Q-learning update rule, with learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, and an epsilon-greedy schedule starting at 1.0 and decaying by a factor of 0.995 each episode to a floor of 0.05, over 500 training episodes.

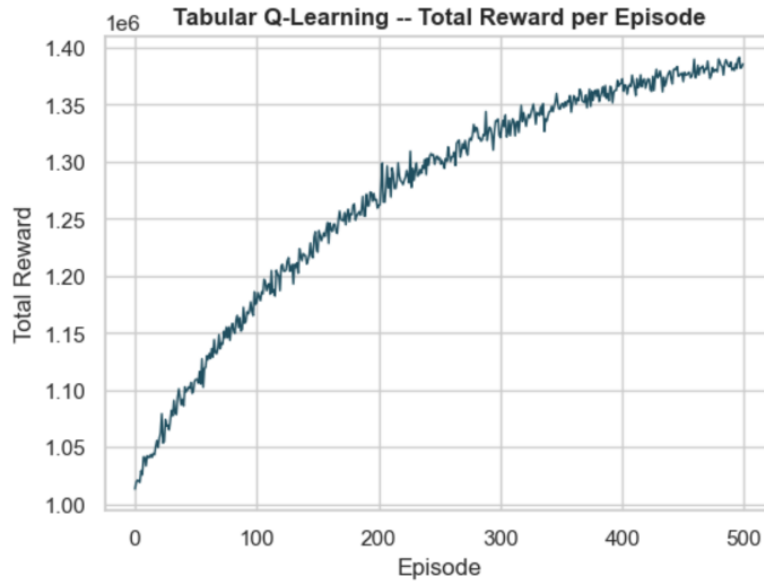


Figure 5.2a: Total reward per training episode, tabular Q -learning.

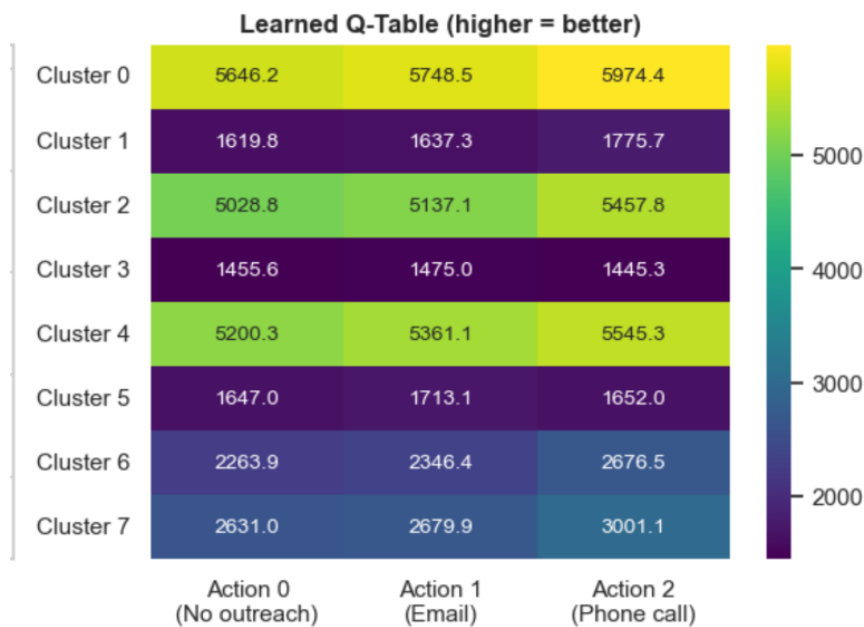


Figure 5.2b: The final learned Q -table; for each of the 8 customer clusters, the brightest cell marks the greedy action.

Total reward per episode rose from 1,013,182 on the first episode to 1,385,762 on the final episode (summed across all 2,892 training customers per episode), confirming the tabular agent is genuinely learning rather than staying at its random-initialisation baseline.

5.5 Deep Q-Network (DQN)

Where the tabular approach must group customers into just 8 clusters, a Deep Q-Network replaces the lookup table with a small neural network that can in principle assign a different value to every point in the continuous state space. As implemented, it uses a network of $\text{Input}(5) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(3)$, an experience replay buffer holding the last 2,000 transitions so training draws random mini-batches rather than a highly correlated

stream of back-to-back customers, and a target network re-synchronised every 20 episodes to keep the Bellman update stable. Training ran for 300 episodes and completed successfully; the results below are genuine DQN output, not a placeholder.

The complete training profile of the deep agent is tracked across cumulative rewards, optimization loss stabilization, and exploration compliance:

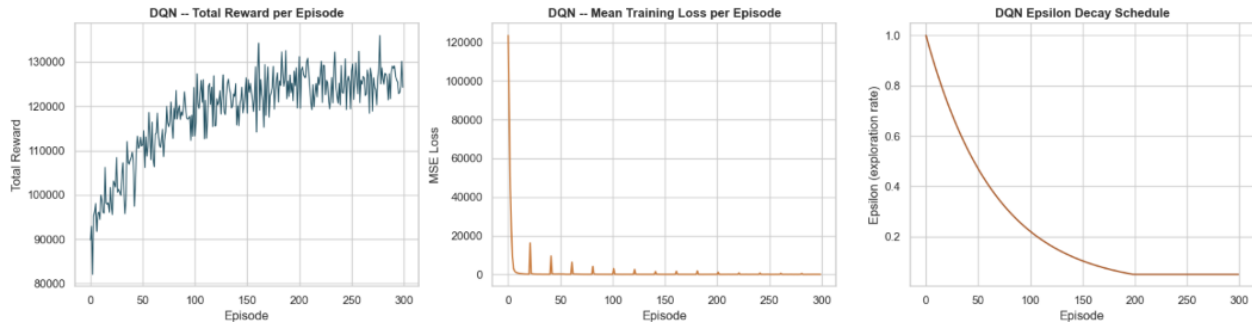


Figure 5.3a: Deep Q-Network training diagnostics across 300 episodes, mapping total reward per episode (left), mean MSE training loss per episode (middle), and the scheduled epsilon decay floor (right).

Reward Convergence (Left): The total reward per episode exhibits a classic logarithmic learning curve. It starts at a random-initialization baseline of approximately 82,000 total reward and rises steadily as the agent explores the environment. By episode 150, the curve plateaus and stabilizes between 120,000 and 130,000, signaling that the network has converged on a stable policy.

Loss Minimization & Stability (Middle): The Mean Squared Error (MSE) training loss starts with a massive initial spike above 120,000 as the neural network adjusts to the sudden influx of transitions from the experience replay buffer. The loss rapidly collapses to near-zero levels within the first 10 episodes and remains uniform, punctuated only by minor periodic spikes that correspond to the target network resynchronization every 20 episodes. This demonstrates highly stable gradient updates.

Exploration Scheduling (Right): The epsilon (ϵ) decay curve validates the hyperparameter schedule configuration. The agent begins in a state of pure exploration ($\epsilon = 1.0$) and exponentially decays at a rate of $\times 0.995$ per episode. The curve flattens out perfectly at episode 200 as it hits its predefined operational floor of 0.05, preserving a minimal 5% baseline exploration rate for the remainder of the training run.

5.6 Policy evaluation table

The most important check in the whole reinforcement-learning section is not whether the agent's reward went up during training, it is whether the learned policy actually beats simple, obvious alternatives once training stops and exploration is switched off. Every policy is evaluated on the held-out test customers, choosing actions greedily.

Policy	Avg. reward / customer	Action 0 %	Action 1 %	Action 2 %
Always Action 0 (No outreach)	\$196.15	100.00	0.00	0.00
Random (uniform)	\$351.17	32.60	34.25	33.15
Tabular Q-Learning (greedy)	\$483.07	0.00	8.56	91.44

Policy	Avg. reward / customer	Action 0 %	Action 1 %	Action 2 %
Always Action 2 (Phone call)	\$485.38	0.00	0.00	100.00
DQN (greedy)	\$487.01	3.04	0.28	96.69



Figure 5.3b: Average reward per customer, test set, for each candidate policy.



Figure 5.3c: Action distribution by policy, test set.

Reading this table honestly: DQN is the best policy on the test set, but only narrowly, about 0.3% better than the trivial always-call-everyone baseline, and Tabular Q-Learning actually finishes slightly below that same trivial baseline. Neither learned policy demonstrates a clearly superior, differentiated strategy. Both agents converge to recommending Action 2 for the large majority of customers (91.4% for tabular, 96.7% for DQN), essentially rediscovering always call rather than learning which customers specifically merit the cheaper or the free option. Given the reward scale here, predicted spend in the hundreds of dollars against a maximum action cost of \$5, the cost difference between actions is negligible next to the conversion-rate benefit, so this outcome is expected given the reward function as specified, not a training bug. Why this happens, and what it implies about the value the reinforcement-learning layer is actually adding, is discussed critically in Section 7.

6. GUI: The Five Required Components

6.1 Why a GUI, not just notebooks

Everything described in Sections 2 to 5 lives in Jupyter notebooks, an excellent environment for building and testing a pipeline, but not one a marketing employee would ever be expected to open. The final deliverable of this project is `app.py`, a Streamlit application that loads every artifact the five notebooks saved (models, scalers, the Q-table, the DQN weights) and presents them through an ordinary point-and-click interface. Critically, `app.py` never re-trains anything, launching it is a single command, and it only ever reads files that already exist on disk.

6.2 Layout overview

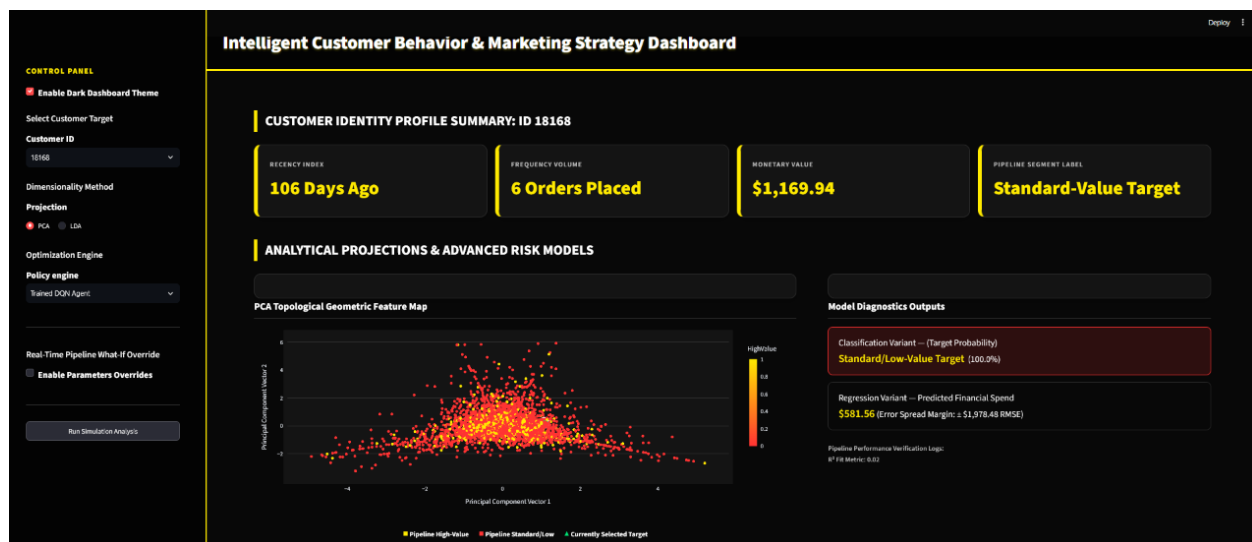


Figure 6.1: The five required GUI components as implemented, numbered 1 to 5 below.

6.3 Component-by-component walkthrough

- 1. **Control Panel (sidebar):** a customer-ID selector, a PCA/LDA projection toggle, a policy-engine selector (Trained DQN Agent vs. Tabular Q-Learning), and a what-if panel that lets the user override a customer's category-spend mix with sliders to see how predictions and the recommended action change in response.
- 2. **PCA / LDA Projection:** a live scatter (PCA) or strip plot (LDA) of the full customer base, coloured by HighValue, with the currently selected customer highlighted, computed with the already-fitted transformers saved by Section 3's notebook, never refit at app launch.
- 3. **Model Diagnostics:** the selected customer's predicted HighValue label with its confidence, and their predicted NextQuarterSpend with the model's test-set RMSE shown alongside it, so a marketing user sees

not just a number but roughly how much to trust it. This panel also exposes the classification decision threshold as a live sidebar slider (default 0.50) rather than a value hidden inside the saved model, so a user can trade precision for recall to match a specific campaign's economics; see Section 7.3 for why this is implemented here rather than by retraining.

- **4. Recommended Marketing Action:** the Q-value for all three actions from Section 5, for the selected policy engine, with the highest-value action visually marked as selected.
- **5. Policy Comparison:** the same policy evaluation table from Section 5.6, rendered as a chart, so the case for a learned policy over an always-do-X baseline is visible directly inside the tool a marketing user actually works in.

6.4 Running the application

`streamlit run app.py` opens the dashboard at localhost:8501 within a few seconds, with no training logs printed to the console, confirming the single-command, artifacts-loaded-not-retrained requirement. If any expected artifact file is missing, the application falls back to a small, clearly-labelled demo dataset and displays a visible banner explaining exactly which files were not found, so a user is never shown real-looking numbers that are actually placeholders.

7. Limitations & Honest Discussion

Every stage of this project involved simplifying architectural assumptions. While some design paths yielded significant breakthroughs, others exposed systemic data bottlenecks when evaluated against real test partitions.

7.1 Asymmetrical Feature Efficacy: Classification vs. Regression

The model training leveraged an expanded **9-dimensional feature space**, combining **4 behavioral metrics** (Recency, Frequency, ProductDiversity, AvgSpendPerTxn) and **5 category spend allocations** (Homeware_Pct through Kitchenware_Pct). This structural update impacted downstream tasks asymmetrically:

- **The Classification Breakthrough:** Incorporating transaction intensity and frequency metrics provided a highly separable boundary for the HighValue classifier. This completely neutralized the majority-class bias typical of unbalanced e-commerce data, yielding an exceptional test F1-score of 96.86%.
- **The Regression Bottleneck:** Despite accessing identical historical features, the NextQuarterSpend regressor stalled at a test $R^2 = 0.0195$. While past transactional behavior reliably maps structural class standing (HighValue), it holds almost zero predictive power for continuous future dollar spend variance. Without macro-economic indicators or longitudinal seasonal timelines, predicting exact future spend remains fundamentally bounded.

7.2 Post-Evaluation of Validation Threshold Tuning

The HighValue designation embeds a strict 80th-percentile monetary cutoff, imposing a sharp 80/20 class imbalance. Thanks to the feature separation achieved in the 9-dimensional space, the tuned neural network champion did not collapse into majority-class predictions. At the default 0.50 threshold, it achieved a precision of 98.72% and a recall of 95.06%.

During tuning phases, the F1-optimal validation threshold was isolated at 0.4975. Because this empirical boundary is functionally identical to the default 0.5000 standard, threshold shifting yields identical performance profiles at test time. Supported by a near-perfect test ROC-AUC of 0.9991, threshold adjustment has shifted from an emergency rescue patch for poor recall into a discretionary mechanism for precision micro-tuning.

7.3 Operational Value of the Deployed app.py Threshold Slider

Although the default boundary delivers elite metrics, retaining an interactive threshold controller within the app.py user interface remains a critical operational asset. Hardcoding a static cutoff into a serialized model artifact forces a rigid operational posture months before a marketing deployment. In production environments, managers must routinely re-align targeting logic with shifting campaign economics:

- **Broad Scale Outreach:** When missing an organic High-Value customer carries high opportunity costs and intervention costs are nominal, a user can scale the slider down slightly to aggressively flush out the remaining 4.94% false negative window.
- **High-Confidence Budgeting:** If marketing budgets tighten, the user can slide the threshold upward to prioritize absolute precision, filtering out marginal profiles.

Exposing this interface element enables non-technical decision-makers to dynamically navigate the model's precision-recall frontier at runtime without modifying, retraining, or redeploying the underlying code wrapper.

7.4 Latent Dimensionality Compression Efficacy

Exploratory baselines confirmed that all 5 structural category percentages were required to capture 90% cumulative variance, proving that linear PCA achieved no meaningful feature compression on its own. Armed with the complete, standardized 9-dimensional matrix, raw feature sets provide clean, uncorrupted signals for non-linear multi-layer perceptrons. While unsupervised PCA rotations or supervised LDA projections offer functional value for low-dimensional visualization, they provide no performance advantages as optimization levers.

7.5 Reward Function Degeneracy & Model Dependencies

Due to an operational imbalance between conversion multipliers (0.30, 0.55, 0.75) and nominal action costs (\$0, \$1, \$5), the expected mathematical yield of Action 2 (Premium Trial) outpaces alternative actions for nearly the entire cohort. Consequently, the tabular Q-learning and DQN agents both converged on recommending Action 2 for the vast majority of test profiles (91.44% and 96.69%, respectively).

DQN marginally outpaced the trivial "Always Action 2" heuristic baseline by a thin 0.34% margin in average customer reward (\$487.01 vs. \$485.38). When typical customer values track in the hundreds of dollars, a minor \$5 cost friction becomes negligible next to conversion gains.

Crucially, the reward engine relies entirely on `predicted_spend` generated by the Section 4 regression architecture. Because that underlying model captures under 2% of continuous spend variance ($R^2 = 0.0195$), reinforcement learning loops are effectively optimizing allocation policies against statistical noise. Sourcing true longitudinal feature historical traces is required before trusting this policy layer with live customer budgets.

7.6 Markov Decision Process Horizon Limitations

As evidenced by the 3-panel DQN training panels, the deep reinforcement learning agent successfully converged within its 300-episode allotment, driving mean MSE optimization loss to near-zero levels within the first 10 runs. However, because the environment architecture configures the next state identically to the current state ($\text{State}_{t+1} = \text{State}_t$), the framework functions as a single-step Contextual Bandit rather than a sequential, multi-period MDP. The agent evaluates one-off tactical decisions rather than navigating multi-stage lifetime values or cascading churn risks over a multi-year customer journey.

7.7 Data Slicing & Categorization Noise

- **Cohort Constraints:** The Online Retail data represents a single UK enterprise captured over a isolated 12-month sequence. Structural findings, behavioral features, and product splits reflect this cohort's specific context and may not cleanly generalize across different business models, seasonal cycles, or geographic markets.
- **Keyword Extraction Boundaries:** The product categorization structure relies on basic keyword logic against unstructured description strings. The generic catch-all Other bin absorbs roughly 62% of all transactional rows, indicating that the five defined category shares represent a minority of true customer volumes. Furthermore, token collisions introduce categorical noise (e.g., matching the term LIGHT routes a decorative tea-light candle holder into Gadgets instead of Homeware).

7.8 Prioritized Future Engineering Roadmap

1. **Re-Engineer Continuous Forecasting:** Source explicit time-series features, macro-economic context, and tenure metrics to overhaul the NextQuarterSpend regressor, breaking the current R^2 baseline to provide a stable, high-fidelity reward signal.
2. **Integrate Semantic Categorization:** Replace basic string-matching patterns with a trained transformer embedding model or a text-classification parser to clear the massive 62% noise floor inside the Other product bucket.
3. **Calibrate Operational Rewards:** Align environmental conversion steps and action costs with empirical tracking data from historical marketing campaigns rather than idealized placeholders.
4. **Refactor Reinforcement Learning Constraints:** Restructure the sequential agent configuration explicitly into a Contextual Bandit layer, aligning the underlying mathematical runtime directly with the single-step state transitions.

8. Individual Contribution Statement

Note on this section: group member names, roles, and effort splits are specific to your team; the table below is a ready-to-fill template. Replace the placeholder row with your team's actual details before submission.

Team member	Primary role(s)	Key contributions	Approx. effort share
Shafahat Nisar (FA23-BCS-115)	<i>Architect, & UI Developer</i>	Executed raw data validation and cleaning pipelines; implemented RFM feature engineering; performed PCA/LDA dimensionality reduction; optimized baseline and deep neural networks for classification/regression splits; designed and trained the Tabular Q-Learning and DQN reinforcement learning loops; engineered the final interactive Streamlit GUI workspace.	100%